

SYSTÈME ET RÉSEAUX : TP10 QUIZZ PROTOCOL
avril 2025 — Pierre Rousselin

Ce TP (qui devrait occuper plusieurs séances) est tiré du partiel 2021–2022.

Exercice 1 : Client et serveur « Quizz Protocol »

Le but de cet exercice est de créer pas-à-pas un client et un serveur implémentant le *Quizz Protocol* décrit plus loin.

- Le *Quizz Protocol* est au-dessus de TCP et est associé au port 6666.
- Le serveur dispose d’une banque de question et lorsqu’un client s’y connecte, le serveur lui envoie une question aléatoire sous la forme d’un énoncé et de plusieurs propositions.
- Le client reçoit l’énoncé de la question et les propositions et les affiche sur le terminal pour l’utilisateur.
- L’utilisateur répond par un numéro (1, 2, ...) de proposition qui est ensuite transmis au serveur (éventuellement modifié de façon à ce qu’il soit compris par le serveur).
- Le serveur, ayant reçu la réponse du client, lui envoie 0 si la réponse est incorrecte ou 1 si c’est la bonne réponse.

Voici deux exemples de déroulement, côté client, en supposant que 127.0.0.1 soit bien l’adresse d’un serveur *Quizz Protocol* :

```
$ quizz/client_quizz 127.0.0.1
9a:1b:c2:70:05:9f peut être une adresse :
1. MAC
2. IPv4
3. IPv6
4. Aucune des autres réponses
5. postale
6. email
Votre réponse : 1
Bravo, c'est la bonne réponse !
$ quizz/client_quizz 127.0.0.1
Que signifie UDP ?
1. Unspecified Data Protocol
2. Union des Démocrates Populaires
3. User Datagram Protocol
4. Unordered Distribution Protocol
Votre réponse : 2
Désolé, ce n'est pas la bonne réponse.
```

Les fonctions main du client et du serveur sont les suivantes :

```
/* client_quizz.c */
int main(int argc, char *argv[])
{
    if (argc != 2) {
        usage(argv[0]);
        return 1;
    }
    int sock = connection_au_serveur(argv[1]);
    echanger_avec_serveur(sock);
    close(sock);
    return 0;
}

/* serveur_quizz.c */
int main(void)
```

```

{
    init_banque_questions();
    int sock = init_sock_ecoute();
    for (;;) {
        int se = connection_au_client(sock);
        if (se < 0)
            continue;
        echanger_avec_client(se, question_aleatoire());
        close(se);
    }
    /* On n'y arrive jamais normalement. */
    close(sock);
    detruire_banque_questions();
    return 0;
}

```

Avant de détailler plus précisément le protocole applicatif, nous commençons par l'établissement de la connection.

0. Préliminaire : Télécharger le dossier associé à ce TP sur *moodle*. Lire les prototypes et spécifications des fonctions du fichier (maison, non standard) `sys_utils.h` écrit en annexe. Vous pouvez utiliser les fonctions de `sys_utils.h` dans tout l'exercice pour gagner du temps (mais ce n'est pas une obligation). *La fonction `exact_read` était à implémenter lors du partiel 2021 – 2022, alors assurez-vous que vous la comprenez bien !*

1. Écrire la définition de la fonction suivante dans le fichier `client_quizz.c`.

```

/** Essayer de se connecter en TCP à l'adresse IPv4 donnée en argument, au port
 * associé au Quizz Protocol.
 * En cas de succès, retourne le descripteur de fichier associé à la socket
 * d'échange avec le serveur.
 * En cas d'échec, met fin au programme après avoir affiché un message d'erreur
 * sur la sortie standard des erreurs. */
int connection_au_serveur(const char *addr_ipv4);

```

Correction de l'exercice 1 :

```

int connection_au_serveur(const char *addr_ipv4)
{
    /* 1. Création d'une socket tcp ipv4 */
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0)
        perror_exit("socket");

    /* 2. Création de la sockaddr distante */
    struct sockaddr_in sa = { .sin_family = AF_INET,
                              .sin_port = htons(PORT_QUIZZP) };
    if (inet_pton(AF_INET, addr_ipv4, &sa.sin_addr) != 1) {
        fprintf(stderr, "adresse ipv4 non valable\n");
        exit(1);
    }

    /* 3. Tentative de connection */
    if (connect(sock, (struct sockaddr *) &sa, sizeof(sa)) < 0)
        perror_exit("connect");
    return sock;
}

```

--- * ---

2. Écrire la définition de la fonction suivante dans le fichier `serveur_quizz.c`

```
/** Créer et configure une socket d'écoute TCP sur IPv4 sur le port associé au
 * Quizz Protocol. Retourne le descripteur de fichier de la socket ainsi créée
 * en cas de succès, met fin au programme sinon. */
```

```
int init_sock_ecoute();
```

Correction de l'exercice 1 :

```
int init_sock_ecoute()
{
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0)
        perror_exit("socket");

    struct sockaddr_in sa = { .sin_family = AF_INET,
                              .sin_port = htons(PORT_QUIZZP),
                              .sin_addr = { htonl(INADDR_ANY) } };
    /* Optionnel : faire en sorte de pouvoir réutiliser l'adresse sans
     * attendre après la fin du serveur. */
    int opt = 1;
    setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(int));
    /* Attacher la socket */
    if (bind(sock, (struct sockaddr *) &sa, sizeof(sa)) < 0) {
        perror("bind");
        exit(2);
    }
    if (listen(sock, 128) < 0) {
        perror("listen");
        return -1;
    }
    return sock;
}
```

--- * ---

3. Écrire la définition de la fonction suivante :

```
/** Accepter les demandes de connection entrantes sur la socket d'écoute
 * sock.
 * Retourne le descripteur de fichier de la socket d'échange créée ou -1 en cas
 * d'erreur */
```

```
int connection_au_client(int sock);
```

Bonus : Faire en sorte que le serveur affiche sur le terminal l'adresse IP du client auquel il est connecté (utiliser les paramètres de l'appel système `accept`, la fonction `inet_ntop` et la constante symbolique `INET_ADDRSTRLEN`).

Correction de l'exercice 1 :

```
int connection_au_client(int sock)
{
    int s;
    struct sockaddr_in addr_clt;
    socklen_t taille_addr = sizeof(addr_clt);

    /* Appel système bloquant accept */
    s = accept(sock, (struct sockaddr *) &addr_clt, &taille_addr);
    if (s == -1) {
        perror("accept");
    } else {
        char str_addr[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &addr_clt.sin_addr.s_addr, str_addr, INET_ADDRSTRLEN);
        printf("Connection au client %s.\n", str_addr);
    }
}
```

```

        return s;
    }

    --- * ---

```

4. Juste pour tester, avant de passer aux choses sérieuses, définir les fonctions `echanger_avec_client` et `echanger_avec_serveur` de façon à ce que :

- Une fois la connection établie, le client envoie "plop !" (terminée par un octet nul) au serveur.
- Le serveur, après avoir reçu ce message (on suppose qu'il connaît son nombre d'octets), l'écrit sur sa sortie standard et répond "hop !" (terminée par un octet nul) au client.
- Après avoir reçu le message du serveur (dont le nombre d'octets est supposé connu à l'avance) et l'avoir écrit sur sa sortie standard, le client répond "Lol !" (terminée par un octet nul) puis ferme sa connexion.
- Le serveur reçoit ce message (encore une fois en connaissant son nombre d'octets), l'écrit sur sa sortie standard et ferme sa connexion à son tour.

Correction de l'exercice 1 :

```

/* Dans serveur_quizz.c */
void echanger_avec_client_test(int fd, const struct question *q)
{
    char buffer[16];
    exact_read(fd, buffer, 7);
    printf("%s\n", buffer);
    strcpy(buffer, "hop !");
    write(fd, buffer, 6);
    exact_read(fd, buffer, 6);
    close(fd);
    printf("%s\n", buffer);
}

/* Dans client_quizz.c */
void echanger_avec_serveur_test(int sock)
{
    char buffer[16];
    strcpy(buffer, "plop !");
    write(sock, buffer, 7);
    exact_read(sock, buffer, 6);
    printf("%s\n", buffer);
    strcpy(buffer, "Lol !");
    write(sock, buffer, 6);
    close(sock);
}

    --- * ---

```

Avant de passer à la suite, il faut détailler le protocole et les données. Le type suivant est défini dans le fichier `question.h` :

```

struct question {
    char *enonce;
    char **propositions;
    uint16_t nb_props;
    uint16_t ind_rep;
    /* ... */
};

```

Le champ `enonce` contient l'adresse d'une chaîne de caractère terminée par `'\0'` qui est l'énoncé de la question. Le champ `propositions` contient l'adresse d'un tableau de propositions, chaque proposition étant l'adresse d'une chaîne de caractère terminée par `'\0'`. Les champs `nb_props` et `ind_rep` sont,

respectivement, le nombre de propositions dans le tableau et l'indice de la bonne réponse dans le tableau (en commençant la numérotation à 0).

Le serveur dispose d'une banque de questions fixée, initialisée au démarrage du serveur. Le type de cette banque défini aussi dans `question.h` est

```
struct banque_questions {
    struct question **tab; /* tableau de nb_questions questions */
    int nb_questions;
};
```

Les fonctions suivantes sont déjà écrites dans `question.c` et vous n'avez pas à vous en soucier¹ :

```
/** Créer et initialiser une banque de question avec les questions du fichier
 * fichier_question.
 * Retourne l'adresse de la banque de questions ainsi créée (en cas d'erreur,
 * met fin au programme). */
struct banque_questions *init_banque_questions(const char *fichier_questions);

/** Retourner une question choisie uniformément au hasard dans la banque de
 * questions d'adresse bq */
struct question *question_aleatoire(const struct banque_questions *bq);

/** Libérer toute la mémoire associée à la banque de questions d'adresse bq */
void detruire_banque_questions(struct banque_questions *bq);
```

Ensuite, l'échange se produit de la façon suivante :

- Le serveur commence par envoyer le nombre de propositions sur 2 octets, en *Network Byte Order*.
- Puis il envoie, à la suite, l'énoncé (avec son `'\0'` final) et les propositions, dans l'ordre, chacune avec son `'\0'` final.
- Le client lit toutes ces informations, affiche énoncé et propositions numérotées sur le terminal (en commençant la numérotation à 1) puis attend la réponse de l'utilisateur. Enfin, il transmet cette réponse moins un, sur deux octets, en *Network Byte Order* au serveur.
- Le serveur lit la réponse de l'utilisateur et la compare à la bonne réponse. Il envoie enfin un seul octet au client, égal à 0 si la réponse est incorrecte et à 1 si c'est la bonne réponse.

Le schéma de la page suivante résume ce protocole applicatif très simple.

1. Conseil : lire et comprendre ces fonctions chez vous.

Le Quizz Protocol

glorieux

Serveur

(processus)

Base de données de questions

hors protocole
(ça ne regarde que le serveur)

(processus)

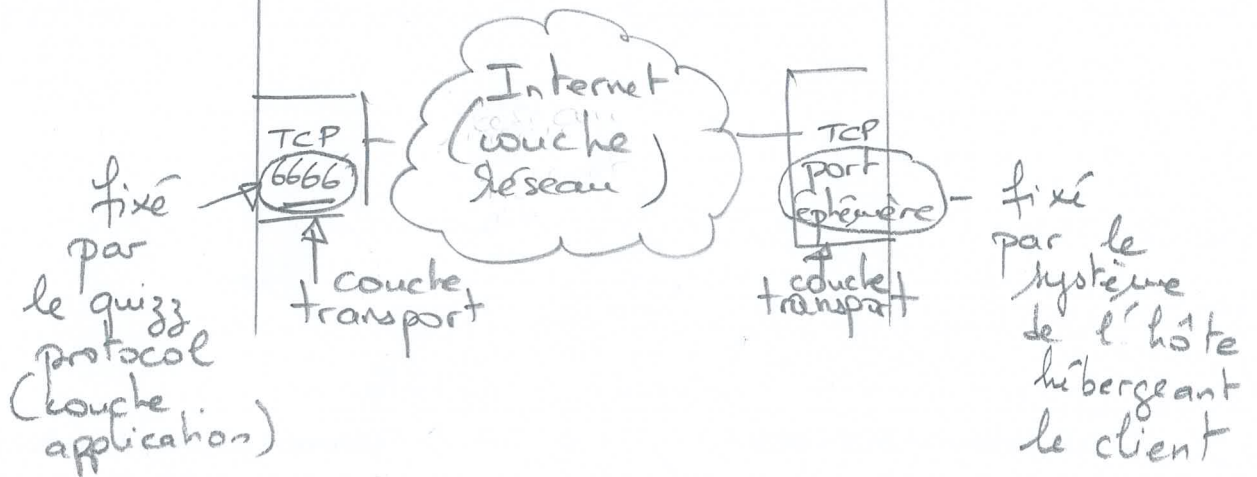
Client

Utilisateur

hors protocole
(ça ne regarde que le client)

couche application:
le Quizz Protocol

n (nombre de propositions)
sur 16 bits, non signé, en big-endian
énoncé : chaîne de caractères
en utf-8 terminée par $\backslash 0$
proposition[0] : idem
:
proposition[$n-1$] : idem
:
réponse proposée : sur 16 bits,
non signé, en big-endian, entre 0 et $n-1$
réponse correcte (1) ou non (0)
sur un seul octet



- 5.a) Combien de propositions différentes peut-il y avoir au maximum dans une question ?
 b) Qu'est-ce que le *Network Byte Order* ?
 c) À quoi sert le caractère `'\0'` à la fin de l'énoncé de la question ?
 d) Pourquoi faut-il envoyer le nombre de propositions ?

Correction de l'exercice 1 :

- a) $2^{16} - 1 = 65535$
 b) L'ordre des octets pour l'écriture des octets dans le réseau, il est gros-boutiste et l'octet de poids fort est en premier.
 c) Il délimite l'énoncé pour le client qui va lire et lui informe que l'énoncé est entièrement lu.
 d) Sinon le client ne peut pas savoir quand s'arrêter de lire des propositions.

--- * ---

6. Écrire la définition de la fonction (côté serveur) :

/ Mener tous les échanges avec le client connecté à la socket fd, ainsi que
 * spécifié dans le Quizz Protocol, avec la question d'adresse q */*

void echanger_avec_client(int fd, const struct question *q);

qui mène tous les échanges décrits précédemment, côté serveur, étant donnés le descripteur de fichier fd d'une socket d'échange et l'adresse q d'une question.

Correction de l'exercice 1 :

```
void echanger_avec_client(int fd, const struct question *q)
{
    /* 1. Écriture du nombre de propositions */
    uint16_t nb_props = htons(q->nb_props);
    if (write(fd, &nb_props, sizeof(nb_props)) < 0)
        perror_exit("write");
    /* 2. Écriture de l'énoncé */
    if (write(fd, q->enonce, strlen(q->enonce) + 1) < 0)
        perror_exit("write");
    /* 3. Écriture des propositions */
    for (uint16_t i = 0; i < q->nb_props; ++i)
        if (write(fd, q->propositions[i], strlen(q->propositions[i]) + 1) < 0)
            perror_exit("write");
    /* 4. Lecture bloquante de la réponse du client */
    uint16_t rep_clt;
    if (exact_read(fd, &rep_clt, sizeof(rep_clt)) < 0)
        perror_exit("read");
    /* 5. Envoi de la réponse du serveur */
    rep_clt = ntohs(rep_clt);
    uint8_t bonne_rep = (rep_clt == q->ind_rep);
    if (write(fd, &bonne_rep, 1) < 0)
        perror_exit("write");
}
```

--- * ---

7. Écrire la définition de la fonction (côté client) :

```
/** Échanger avec le serveur connecté via la socket sock ainsi que spécifié par  

  * le protocole Quizz Protocol, tout en menant les échanges avec l'utilisateur.  

  *  

  * - Imprimer sur la sortie standard la question et les propositions envoyées  

  *   par le serveur  

  * - Demander à l'utilisateur un entier entre 1 et le nombre de propositions  

  * - Envoyer le numéro de la réponse choisie au serveur  

  * - Recevoir le message final du serveur (réponse correcte ou incorrecte)  

  * - Dire à l'utilisateur si sa réponse est correcte ou non  

  */
void echanger_avec_serveur(int sock);
```

qui mène tous les échanges décrits précédemment, côté client, étant donnés le descripteur de fichier `fd` d'une socket d'échange connectée avec le serveur.

Il faut que chaque proposition soit convenablement numérotée (en partant de 1).

Vous pourrez utiliser la fonction suivante (définie dans `sys_utils.c`) :

```
/* Retourner un entier positif saisi par l'utilisateur, sur deux octets, compris
 * entre borne_inf (incluse) et borne_sup (incluse) */
```

```
uint16_t saisir_entre_bornes(uint16_t borne_inf, uint16_t borne_sup);
```

Indication : pour les lectures de chaînes de caractère, il est sans doute plus simple de lire 1 seul caractère à la fois et de l'afficher immédiatement.

Correction de l'exercice 1 :

```
void echanger_avec_serveur(int sock)
{
    /* 1 nombre de propositions */
    uint16_t nb_props;
    exact_read(sock, &nb_props, sizeof(nb_props));
    nb_props = ntohs(nb_props);
    /* 2 énoncé */
    char c;
    while (read(sock, &c, 1) > 0 && c != '\0')
        putchar(c);
    putchar('\n');
    /* 3 propositions */
    for (uint16_t i = 1; i <= nb_props; ++i) {
        printf("%u. ", i);
        while (read(sock, &c, 1) > 0 && c != '\0')
            putchar(c);
        putchar('\n');
    }
    /* 4 réponse utilisateur */
    uint16_t rep_util = saisir_entre_bornes(1, nb_props) - 1;
    rep_util = htons(rep_util);
    if (write(sock, &rep_util, sizeof(uint16_t)) < 0) {
        perror("write");
    }
    /* 5 réponse du serveur */
    uint8_t rep_serv;
    read(sock, &rep_serv, sizeof(rep_serv));
    if (rep_serv == 0)
        printf("Désolé, ce n'est pas la bonne réponse.\n");
    else
        printf("Bravo, c'est la bonne réponse !\n");
}
```

--- * ---

- 8.a) Que se passe-t-il si un deuxième client cherche à se connecter au serveur lorsque l'utilisateur d'un premier client est en train de réfléchir à sa réponse (pendant peut-être longtemps) ? Dans quels appels systèmes sont bloqués le serveur et les deux clients ?

Correction de l'exercice 1 : Le premier client est bloqué dans `read` sur le terminal, le serveur est donc bloqué dans `read` sur sa socket, en attente de la réponse de ce client. Le second client est bloqué dans son `connect`, car le serveur n'écoute pas.

--- * ---

- b) On considère le type structuré suivant :

```
struct work {
    int socket;
```

```

    struct question *q;
};

```

Écrire la définition de la fonction suivante :

```

/* Reçoit l'adresse d'une variable de type struct work dynamiquement allouée sur
 * le tas et convenablement initialisée, puis échange avec un client en
 * utilisant le champ socket de cette structure pour la question (champ q) de
 * cette structure.
 * À la fin des échanges, libère la mémoire associée à l'adresse work.
 * Retourne toujours NULL. */

```

```
void *worker(void *work);
```

Correction de l'exercice 1 :

```

void *worker(void *work)
{
    struct work *w = work;
    echanger_avec_client(w->socket, w->q);
    close(w->socket);
    free(w);
    return NULL;
}

```

--- * ---

- c) Écrire tous les changements dans la fonction main du serveur pour que chaque client soit servi par un *thread* séparé. Faire en sorte que la mémoire occupée par ce *thread* soit libérée lorsqu'il se termine.

Correction de l'exercice 1 :

```

struct work *w = malloc_or_exit(sizeof(struct work));
if ((w->socket = connection_au_client(sock)) < 0)
    continue;
w->q = question_aleatoire();
pthread_t th;
if (pthread_create(&th, NULL, worker, w) < 0)
    perror_exit("pthread_create");
pthread_detach(th);

```

--- * ---

--- * ---

Fonctions utilitaires pour gagner du temps : sys_utils.h

```

/** Allouer nbytes octets et renvoie un pointeur sur la mémoire allouée.
 * En cas d'échec, affiche un message d'erreur et met fin au programme avec
 * statut de sortie 2. */
void *malloc_or_exit(size_t nbytes);

/** Lire exactement nbytes octets (si besoin en appelant plusieurs fois read)
 * dans le fichier de descripteur fd ou bien jusqu'à la fin du fichier et les
 * copier dans buf.
 * Retourne le nombre d'octets lus ou -1 en cas d'erreur. */
ssize_t exact_read(int fd, void *buf, size_t nbytes);

/** Utiliser la fonction perror avec l'argument err_mess et mettre fin au
 * programme avec statut de sortie 2. */
void perror_exit(const char *err_mess);

/** Vider le tampon associé à l'entrée standard */

```

```
void vider_buffer();
```

```
/** Retourner un entier positif saisi par l'utilisateur, sur deux octets, compris
 * entre borne_inf (incluse) et borne_sup (incluse) */
uint16_t saisir_entre_bornes(uint16_t borne_inf, uint16_t borne_sup);
```

Prototypes des fonctions à utiliser, quelques noms de constantes symboliques et quelques champs de types structurés

Certains arguments sont mis à NULL ou 0 car nous n'avons pas vu leur rôle en cours.

```
int getchar();
int putchar(int c);

size_t strlen(const char *s);
char *strcpy(char *dest, const char *src);
void *memcpy(void *dest, const void *src, size_t n);
void *memset(void *s, int c, size_t n);

pid_t fork();
pid_t wait(int *wstatus);
void exit(int status);

int pthread_create(pthread_t *thread, NULL,
                  void *(*start_routine) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
int pthread_detach(pthread_t thread);

int pthread_mutex_init(pthread_mutex_t *mutex, NULL);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);

int sem_init(sem_t *sem, 0, unsigned int valeur);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);

int open(const char *path, int oflag, ...);
ssize_t read(int fildes, void *buf, size_t nbyte);
ssize_t write(int fildes, const void *buf, size_t nbyte);
int close(int fildes);

int socket(int domain, int type, 0);
int bind(int socket, const struct sockaddr *address, socklen_t address_len);
int listen(int socket, int backlog);
int accept(int socket, struct sockaddr *address, socklen_t *address_len);
int connect(int socket, const struct sockaddr *address, socklen_t address_len);

ssize_t recv(int socket, void *buffer, size_t length, int flags);
ssize_t send(int socket, const void *buffer, size_t length, int flags);
ssize_t recvfrom(int socket, void *buffer, size_t length, int flags,
                 struct sockaddr *address, socklen_t *address_len);
ssize_t sendto(int socket, const void *message, size_t length, int flags,
               const struct sockaddr *dest_addr, socklen_t dest_len);
```

```
uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);

const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
int inet_pton(int af, const char *src, void *dst);

struct sockaddr_in {
    sa_family_t    sin_family; /* address family: AF_INET */
    in_port_t      sin_port;   /* port in network byte order */
    struct in_addr sin_addr;   /* internet address */
};

struct in_addr {
    in_addr_t s_addr;
};
```

Quelques constantes symboliques :

- AF_UNIX, AF_INET, AF_INET6
- SOCK_DGRAM, SOCK_RAW, SOCK_STREAM
- INADDR_ANY, IN6ADDR_ANY_INIT
- INET_ADDRSTRLEN, INET6_ADDRSTRLEN
- O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_APPEND, O_TRUNC